

Análisis de Lenguajes de Programación

Sistemas de Tipos

6 de Octubre de 2025

¿Cómo surgen?

- ▶ Los primeros sistemas (rudimentarios) aparecieron con los primeros lenguajes de programación, como Fortran y Cobol. Permitían detectar una variedad de errores en tiempo de compilación, pero tenían deficiencias.
- ▶ En 1970 surgen los algoritmos para inferencia de tipos para eliminar la información de tipos explícita. El algoritmo de **Hindley-Milner** fue usado por 1º vez para ML.
- ▶ Este algoritmo sigue siendo la base para los sistemas de tipos de lenguajes funcionales.
- ▶ Al agregar expresividad al sistema de tipos, la inferencia puede volverse indecidible.

¿Qué son?

Un sistema de tipos es **“un método sintáctico para probar la ausencia de ciertos comportamientos mediante la clasificación de frases de acuerdo a los valores que computan”**. (B. Pierce)

Además de detectar errores en una etapa temprana del desarrollo del software los sistemas de tipos ofrecen otras ventajas:

Sirven también para:

- ▶ Especificación rudimentaria o Documentación (siempre actualizada).
- ▶ Abstracción
- ▶ Optimización
- ▶ Lenguaje seguros

Chequeo estático

- ▶ Los sistemas de tipos proveen un **chequeo estático**, es decir en tiempo de compilación.
- ▶ Al ser estáticos, sólo se puede garantizar la ausencia de ciertos errores.
- ▶ Los sistemas de tipos son necesariamente **conservadores**, permiten probar la ausencia de determinados errores, pero no probar la presencia de errores. Por ej,
`if test then S else (error de tipo)`
es usualmente rechazado aunque test sea siempre verdadero.
- ▶ Los lenguajes con **tipos dependientes** permiten hacer algunas de estas verificaciones estáticamente.

Chequeo dinámico

- ▶ El chequeo del tipo de un programa es realizado en tiempo de **ejecución**.

Por ejemplo, en una expresión de la forma $e_1 + e_2$, primero se evalúan las expresiones y dependiendo del tipo de éstas, se aplica la operación $+$.

- ▶ Algunos lenguajes incluyen un chequeo dinámico, incluso teniendo un sistema de tipos para el chequeo estático.
- ▶ Los lenguajes que incluyen un chequeo dinámico, pero no un sistema de tipos, se denominan “**Lenguajes con tipado dinámico**”.

¿Qué es un lenguaje seguro?

- ▶ Según Pierce: “Un lenguaje es seguro si protege sus abstracciones”
Por ejemplo,
 - ▶ Si un lenguaje provee la abstracción de **arreglo**, para ser seguro debe garantizar que no hay manera de escribir fuera del arreglo.
 - ▶ Un lenguaje con variables locales, deben poder accederse sólo bajo su alcance.
- ▶ Si un lenguaje no es seguro, es difícil entender qué hace un programa. Ejemplos: C, C++, donde el comportamiento de algunos programas que utilizan punteros no puede predecirse.

- ▶ ¡Lenguaje seguro no es igual a tipado estático!
- ▶ Formalizaremos una propiedad de seguridad básica: "Los términos tipados no causan errores".
- ▶ Las formas normales que no son valores se denominan **términos atascados**.
- ▶ El sistema de tipos debe garantizar que los términos bien tipados nunca se atasquen.

Seguridad = Progreso + Preservación

Progreso: Si $t : T$, entonces t es un valor o existe t' tal que $t \rightarrow t'$ (es decir, t no está atascado).

Preservación: Si $t : T$ y $t \rightarrow t'$, entonces $t' : T$.

Ambas propiedades aseguran que ningún término bien tipado se atasca durante su evaluación.

Ejemplo

Sintaxis de lenguaje de expresiones aritméticas y booleanas:

$$\begin{aligned} t &::= \text{true} \\ &| \text{false} \\ &| \text{if } t \text{ then } t \text{ else } t \\ &| 0 \\ &| \text{suc } t \\ &| \text{pred } t \\ &| \text{iszero } t \end{aligned}$$
$$v ::= \text{true} \mid \text{false} \mid nv$$
$$nv ::= 0 \mid \text{suc } nv$$
$$T ::= \text{Bool} \mid \text{Nat}$$

Reglas de tipado

- ▶ Agregamos una relación de tipado $t : T$, se lee " t tiene tipo T ".
- ▶ Decimos que t es tipado si existe T tal que $t : T$.
- ▶ La relación de tipado $t : T$ y se define como la menor relación que satisface las siguientes reglas:

$true : Bool \text{ (T-TRUE)}$

$false : Bool \text{ (T-FALSE)}$

Reglas de tipado

$$\frac{t_1 : Bool \quad t_2 : T \quad t_3 : T}{if \ t_1 \ then \ t_2 \ else \ t_3 : T} \quad (T-IF)$$

$$0 : Nat \quad (T-ZERO)$$

$$\frac{t : Nat}{pred \ t : Nat} \quad (T-PRED)$$

$$\frac{t : Nat}{suc \ t : Nat} \quad (T-SUC)$$

$$\frac{t : Nat}{iszero \ t : Bool} \quad (T-ISZERO)$$

Derivación de tipado

Mostraremos que un término tiene un tipo dado mediante una **derivación de tipado**.

Por ejemplo *if iszero (suc 0) then pred (suc 0) else 0 : Nat* dado que:

$$\frac{\frac{\overline{0 : \text{Nat}} \quad \text{T-ZERO}}{\text{iszero } 0 : \text{Bool}} \quad \text{T-ISZERO} \quad \frac{\overline{0 : \text{Nat}} \quad \text{T-ZERO}}{\text{pred } 0 : \text{Nat}} \quad \frac{\overline{0 : \text{Nat}} \quad \text{T-ZERO}}{\text{pred } 0 : \text{Nat}} \quad \text{T-PRED}}{\text{if iszero } 0 \text{ then } 0 \text{ else pred } 0 : \text{Nat}} \quad \text{T-IF}$$

Reglas de semántica operacional

Definimos la semántica operacional del lenguaje mediante una relación $\rightarrow \in \mathcal{T} \times \mathcal{T}$, definida por las siguientes reglas:

$$\text{if true then } t_2 \text{ else } t_3 \rightarrow t_2 \quad (\text{E-IFTRUE})$$

$$\text{if false then } t_2 \text{ else } t_3 \rightarrow t_3 \quad (\text{E-IFFALSE})$$

$$\frac{t_1 \rightarrow t'_1}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightarrow \text{if } t'_1 \text{ then } t_2 \text{ else } t_3} \quad (\text{E-IF})$$

$$\frac{t \rightarrow t'}{\text{suc } t \rightarrow \text{suc } t'} \quad (\text{E-SUC})$$

Reglas de semántica operacional

$$\text{pred } 0 \rightarrow 0 \quad (\text{E-PREDZERO})$$

$$\text{pred } (\text{suc } nv) \rightarrow nv \quad (\text{E-PREDSUC})$$

$$\frac{t \rightarrow t'}{\text{pred } t \rightarrow \text{pred } t'} \quad (\text{E-PRED})$$

$$\text{iszero } 0 \rightarrow \text{true} \quad (\text{E-ISZEROZERO})$$

$$\text{iszero } (\text{suc } nv) \rightarrow \text{false} \quad (\text{E-ISZEROSUC})$$

$$\frac{t \rightarrow t'}{\text{iszero } t \rightarrow \text{iszero } t'} \quad (\text{E-ISZERO})$$

Teorema (Progreso): Si $t : T$ entonces, t es un valor o existe t' tal que $t \rightarrow t'$.

La demostración es por inducción en la derivación $t : T$.

Caso T-IF Si la última regla aplicada en la derivación $t : T$ es T-IF entonces:

- ▶ $t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3$
- ▶ $t_1 : \text{Bool}$, $t_2 : T$ y $t_3 : T$

Dado que $t_1 : \text{Bool}$, por HI, t_1 es un valor o existe t'_1 tal que $t_1 \rightarrow t'_1$. Si t_1 es un valor, éste es *true* o *false* y puedo aplicar E-IFTrue o E-IFFalse para encontrar t' . Si existe t'_1 tal que $t_1 \rightarrow t'_1$, puedo aplicar la regla E-If.

Cálculo lambda simplemente tipado ($\lambda_{\rightarrow}$)

Términos:

$$t ::= x \mid c \mid t t \mid \lambda x. t$$

Tipos:

$$T ::= B \mid T \rightarrow T$$

donde B es un conjunto de tipos bases y c un conjunto de constantes.

Definimos una familia de cálculos.

Reglas de tipado

- ▶ Al introducir variables al lenguaje, la relación de tipado deberá tener un elemento más: un **contexto de tipado** o **entorno de tipo**.
- ▶ Un entorno de tipos es una secuencia de variables con sus tipos.
- ▶ Algunas convenciones, sea Γ , un entorno de tipos:
 - ▶ El operador $,$ extiende un entorno agregando una variable con su tipo. Por ej: $\Gamma, x : T_1$
 - ▶ El entorno vacío se denota con $\emptyset$, o simplemente se omite.

Más convenciones

- ▶ Para evitar confusiones entre nombres de variables, se supone que al agregar una variable al entorno Γ , ésta no está en Γ , dado que los nombres de las variables ligadas pueden ser renombrados.
- ▶ Γ entonces puede pensarse también como una función de variables a tipos.
- ▶ Escribiremos $\Gamma(x) = T_1$ o $x : T_1 \in \Gamma$ para decir que la variable x tiene tipo T_1 en Γ .

Reglas de tipado

La relación de tipado es ternaria $\Gamma \vdash t : T$, significa que el término t tiene tipo T en el entorno Γ .

$$\frac{x : T \in \Gamma}{\Gamma \vdash x : T} \quad (\text{T-VAR})$$

$$\frac{\Gamma \vdash t_1 : T_1 \rightarrow T_2 \quad \Gamma \vdash t_2 : T_1}{\Gamma \vdash t_1 t_2 : T_2} \quad (\text{T-APP})$$

$$\frac{\Gamma, x : T_1 \vdash t : T_2}{\Gamma \vdash \lambda x. t : T_1 \rightarrow T_2} \quad (\text{T-ABS})$$

Faltan reglas para las constantes.

Determinar el tipo de los siguientes términos dando el árbol de derivación siendo $\Gamma = f : (B \rightarrow B) \rightarrow B \rightarrow B, x : B$:

1. $(\lambda y . y) x$

2. $(\lambda g . f g) x$

- ▶ Algunos términos del λ -cálculo sin tipo no son válidos en $\lambda_{\rightarrow}$, ya que no pueden tiparse, por ejemplo Y .
- ▶ Es imposible encontrar un tipo para cualquier operador de punto fijo.
- ▶ La recursión se puede recuperar agregando un operador **fix**, se agrega como constante.

$$\begin{aligned} \text{fix} &: (T \rightarrow T) \rightarrow T \\ \text{fix } t &= t (\text{fix } t) \end{aligned}$$

- ▶ A diferencia de λ -cálculo sin tipos, $\lambda_{\rightarrow}$ es fuertemente normalizante, dado que cualquier término es fuertemente normalizante.

Estrategias de reducción

- ▶ La estrategia de reducción *call-by-value*, es **estricta**, dado que el argumento de una función se evalúa independientemente si se usa en el cuerpo de la función.
- ▶ Mientras que las estrategias *call-by-name* y *call-by-need* son no estrictas o *lazy*.
- ▶ Notar que el operador de punto fijo Y , no sirve con una estrategia de reducción *call-by-value*, Yg *diverge para cualquier g* .

Daremos una estrategia *call-by-value* para este cálculo:

$$\frac{t_1 \rightarrow t'_1}{t_1 \ t_2 \rightarrow t'_1 \ t_2} \quad (\text{E-APP1})$$

$$\frac{t_2 \rightarrow t'_2}{v \ t_2 \rightarrow v \ t'_2} \quad (\text{E-APP2})$$

$$(\lambda x . t) \ v \rightarrow t \ [v/x] \quad (\text{E-APPABS})$$

donde $v ::= c \mid \lambda x . t$

E-App1 y *E-App2* son reglas de congruencia, mientras que *E-AppAbs* es de computación.

- ▶ $\lambda_{\rightarrow}$ satisface los teoremas de **Progreso** y **Preservación**.
- ▶ Para probar Preservación se utiliza el siguiente lema de sustitución:

$$\frac{\Gamma \vdash s : S \quad \Gamma, x : S \vdash t : T}{t[s/x] : T}$$

Notación explícita de tipo

- ▶ El cálculo que vimos es el $\lambda_{\rightarrow}$ a la Curry, donde por ejemplo el término $\lambda x . x$ es un esquema de términos.
- ▶ La notación que agrega explícitamente el tipo a las variables se denomina a la Church.
Las abstracciones son de la forma $\lambda x : T . t$
- ▶ Los cálculos $\lambda_{\rightarrow}$ a la Curry y la Church son equivalente.

- ▶ El sistema T extiende $\lambda_{\rightarrow}$ con:
 1. Los tipos Bool y Nat.
 2. Constantes para naturales: **0** y **succ** (constructores), **R** (eliminador).
 3. Constantes para booleano: **true** y **falso** (constructores), **D** (eliminador).
- ▶ La constante **D** es la operación if-then-else.
- ▶ La constante **R** es la recursión primitiva para naturales.

Reglas de tipado adicionales

$$\Gamma \vdash \text{true} : \text{Bool} \quad (\text{T-TRUE})$$

$$\Gamma \vdash \text{false} : \text{Bool} \quad (\text{T-FALSE})$$

$$\frac{\Gamma \vdash t_1 : \text{Bool} \quad \Gamma \vdash t_2 : T \quad \Gamma \vdash t_3 : T}{\Gamma \vdash D \ t_1 \ t_2 \ t_3 : T} \quad (\text{T-D})$$

$$\Gamma \vdash 0 : \text{Nat} \quad (\text{T-0})$$

$$\frac{\Gamma \vdash t : \text{Nat}}{\Gamma \vdash \text{succ } t : \text{Nat}} \quad (\text{T-SUCC})$$

$$\frac{\Gamma \vdash t_1 : T \quad \Gamma \vdash t_2 : T \rightarrow \text{Nat} \rightarrow T \quad \Gamma \vdash t_3 : \text{Nat}}{\Gamma \vdash R \ t_1 \ t_2 \ t_3 : T} \quad (\text{T-REC})$$

Reglas de evaluación adicionales

$$D \text{ true } t_2 \ t_3 \rightarrow t_2 \quad (\text{E-DTRUE})$$

$$D \text{ false } t_2 \ t_3 \rightarrow t_3 \quad (\text{E-DFALSE})$$

$$\frac{t_1 \rightarrow t'_1}{D \ t_1 \ t_2 \ t_3 \rightarrow D \ t'_1 \ t_2 \ t_3} \quad (\text{E-D})$$

$$R \ t_1 \ t_2 \ 0 \rightarrow t_1 \quad (\text{E-R0})$$

$$R \ t_1 \ t_2 \ (\text{succ } t) \rightarrow t_2 \ (R \ t_1 \ t_2 \ t) \quad (\text{E-RSUCC})$$

$$\frac{t_3 \rightarrow t'_3}{R \ t_1 \ t_2 \ t_3 \rightarrow R \ t_1 \ t_2 \ t'_3} \quad (\text{E-R})$$

- ▶ Factorial en Haskell:

```
fact 0 = 1
```

```
fact (n+1) = (n + 1) * fact n
```

- ▶ Factorial en Sistema T

$$fact = R\ 1\ (\lambda\ r\ m.\ prod\ (succ\ m)\ r)$$

Definir en Sistema T las siguientes funciones:

1. $pred : Nat \rightarrow Nat$
2. $suma : Nat \rightarrow Nat \rightarrow Nat$
3. $prod : Nat \rightarrow Nat \rightarrow Nat$
4. $isO : Nat \rightarrow Bool$
5. $eq : Nat \rightarrow Nat \rightarrow Bool$